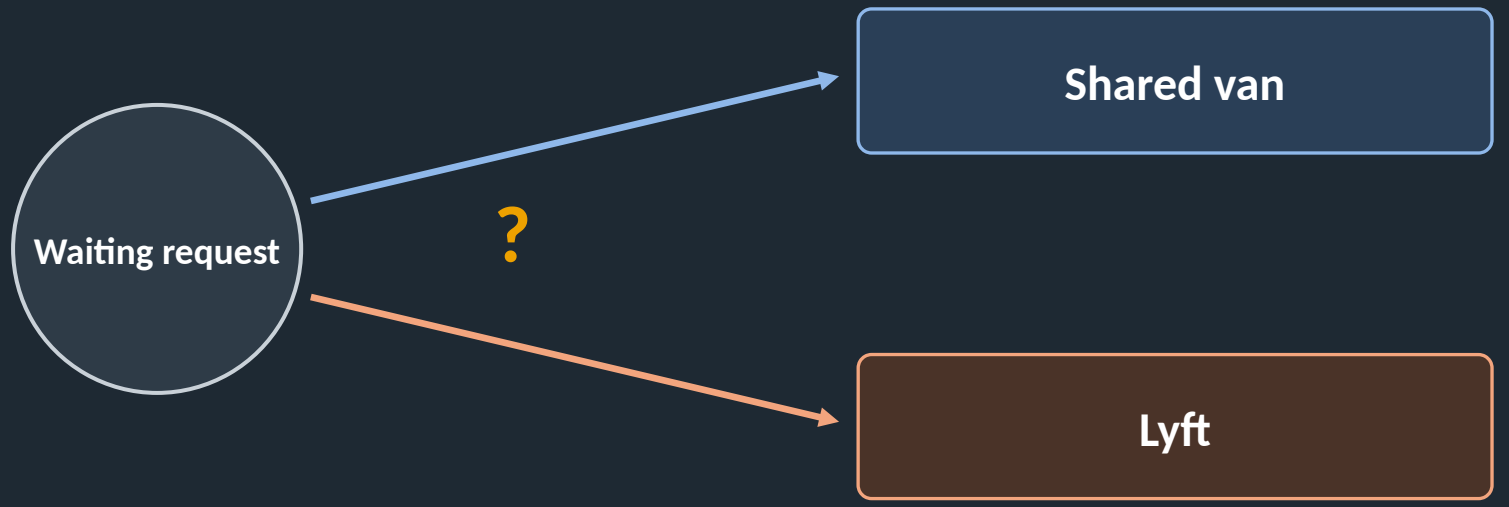
# When should Night Ride move a waiting request to Lyft?

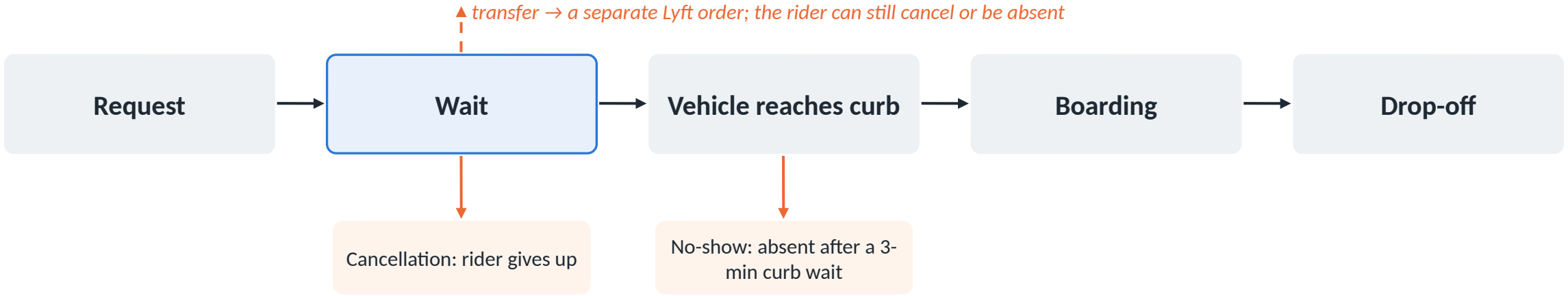
What April's aggregate data helped us build—and what our first simulations revealed.



# The service and one request

**Homewood-area Night Ride pools requests in shared vans. A dispatcher can move waiting riders to Lyft. We are studying when that transfer should happen.**

Area: Homewood Night Ride, including the Peabody area; East Baltimore excluded.




**Booking**
one Night Ride request, possibly with several passengers

**Lyft order**
a separate order placed after a transfer decision

**Curb arrival**
the vehicle reaches the pickup location

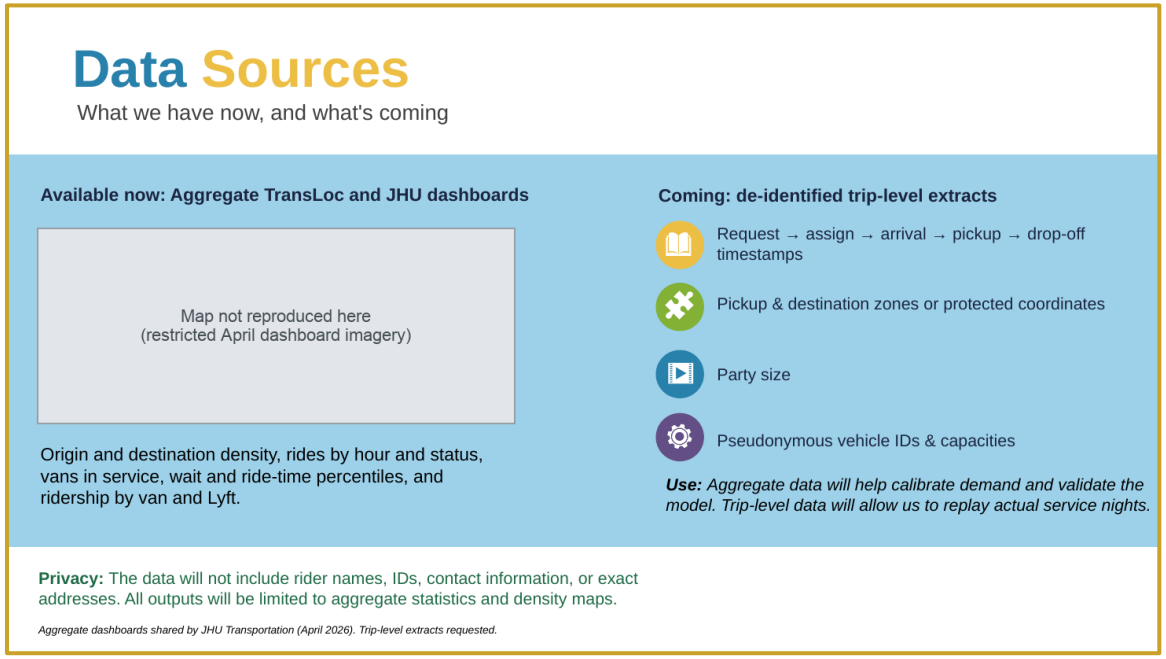
**Pickup**
the rider boards

Waiting time runs from the request to curb arrival. Each Lyft order is an extra trip Hopkins pays for (its cost is unknown to us), so moving riders to Lyft has a price.

Source: docs/technical/decision_log.md D-23, D-27; scenario_spec.md v0.8 §5–§6; JHU Night Ride page (3-min curb wait).

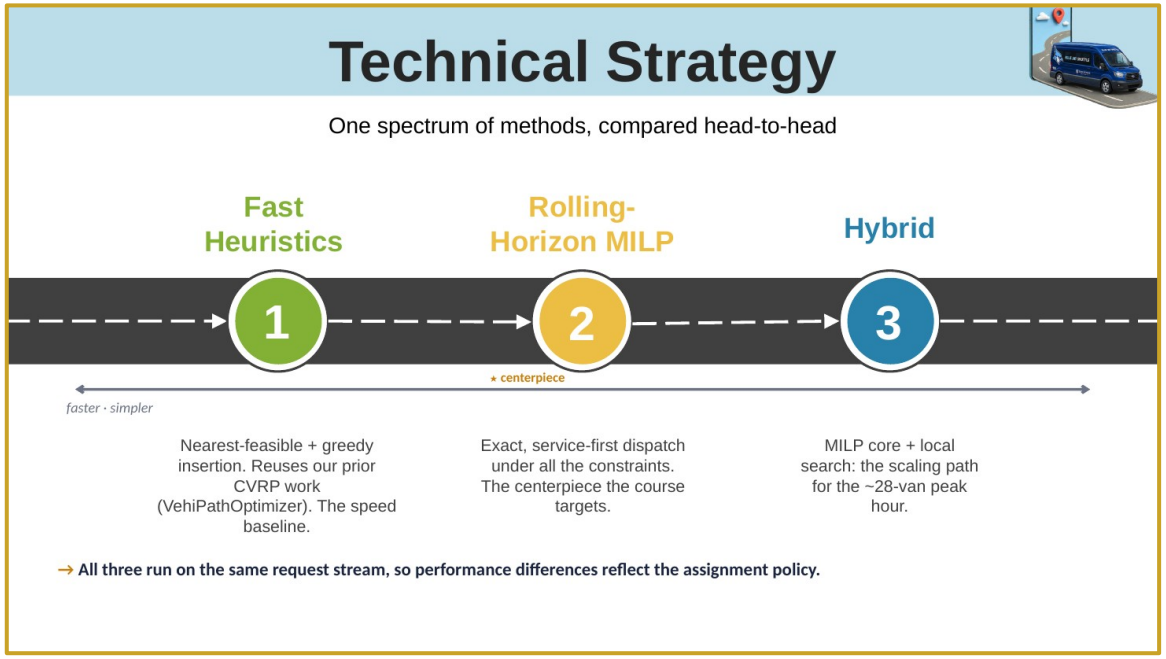
# Where we started

ORIGINAL PROPOSAL




Original proposal, slide 10: trip-level extracts were expected, to replay actual service nights

ORIGINAL PROPOSAL




Original proposal, slide 11: compare heuristics, a rolling-horizon MILP and a hybrid on the same request stream

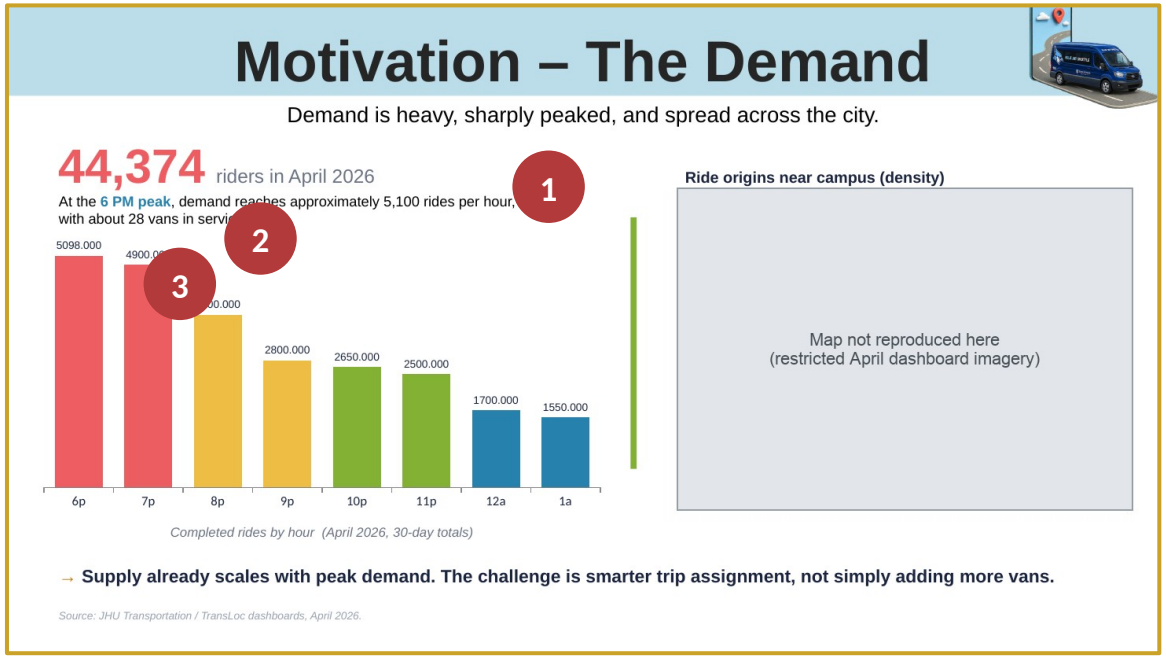
**What changed:** the trip-level extracts never arrived, so real nights could not be replayed. The data we have are April's aggregate dashboards. We also narrowed the question (next slides).

Source: BlueJayNightRide_Proposal.pptx (September, committed version), slides 10–11.

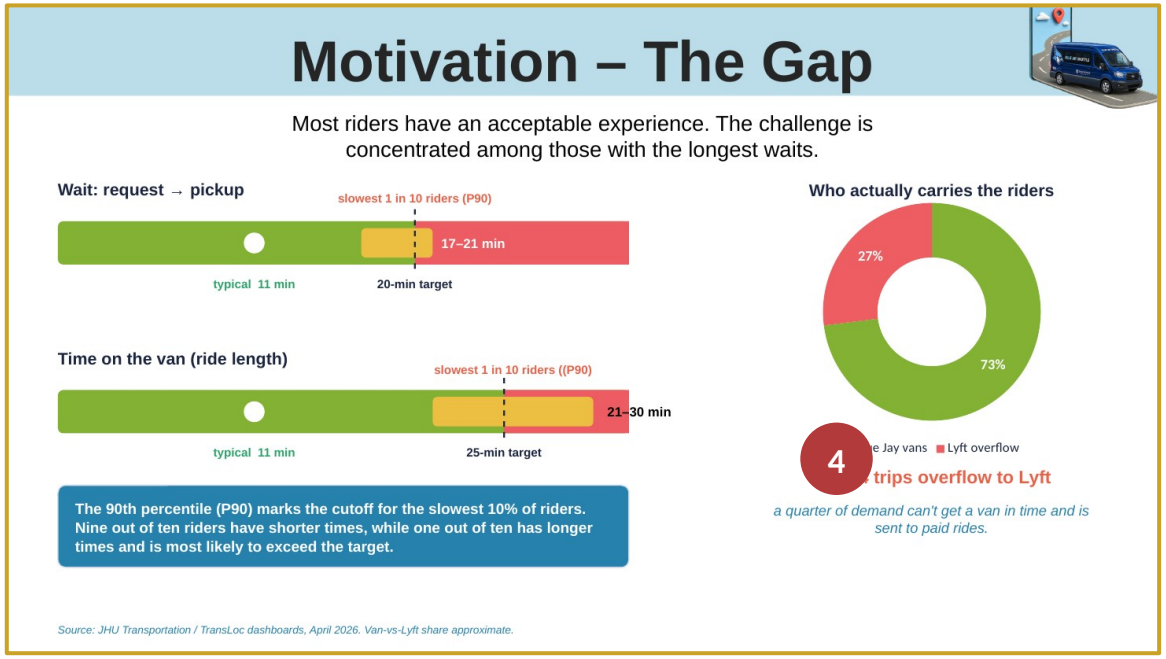
All proposal corrections: B9 →

# Where we started: numbers we later corrected

ORIGINAL PROPOSAL




ORIGINAL PROPOSAL




1 **"about 5,100 rides per hour" at 6 pm** → a 30-day total: about 170 completed rides in one evening's 6 pm hour

2 **"about 28 vans in service"** → the "All services" average (27.6 at 6 pm), not a Homewood-only count

3 **7 pm bar "4,900"** → our digitized value is 3,957 (also a 30-day total)

4 **"~1 in 4 trips overflow to Lyft"** → 24.5% of riders rode Lyft: a share of riders, not of trips or requests

Source: Proposal slides 7–8; April dashboards (digitized aggregates); methodology v1.5 §1a. Details: B9.

Details: B9 →

# We narrowed the question to one controllable decision

**We now hold one specified van dispatcher fixed and change only the Lyft transfer rule.**

## Original ambition

- Replay real service nights
- Compare dispatch methods head to head
- Lyft use decided inside the optimizer itself

## Evidence available

- Aggregate April dashboards only
- No individual journeys
- Hopkins' current dispatcher internals not verified

## Current controlled experiment

- One specified van dispatcher, held fixed
- Change only the Lyft transfer rule
- Same constructed nights for every rule

**This lets us study how the timing and information used by that rule change service and Lyft use.**

Why: it is the question we chose, the real dispatcher's internals are not known to us, and a controlled comparison needs everything else held fixed. Our van dispatcher re-plans every 2 minutes and at each new request, cancellation or no-show while waiting, or van coming on duty: modelled on TransLoc's (the vendor's dispatch software) published 2016 design, not verified current Hopkins practice.

Source: docs/technical/scenario_methodology.md v1.5 §1a; decision_log.md D-25, D-51; transloc_operations.md.

# April describes totals—not individual journeys

**We know how many requests ended in each status, where pickups and drop-offs accumulated, and how activity varied by hour. We cannot reconstruct each rider's journey.**

## Observed: totals and shares

- Requests by final status: 50,297 in the month
- Completed rides by hour (30-day totals)
- Vehicles on duty by hour (27.6 at 6 pm, all services)
- Pickup and drop-off counts at 188 map places
- Wait-time percentiles
- Riders by van and by Lyft (44,374; 24.5% by Lyft)

## Missing: any single journey

- When each request was made
- Which pickup went with which drop-off
- How long that rider waited
- Whether they boarded, gave up or were moved to Lyft

**OBSERVED** read from April's dashboards

**DERIVED** computed from observed numbers

**ASSUMED** our modelling choice, to be tested

**DESIGNED** an experiment setting we chose

*We therefore built plausible evenings from the evidence, while keeping the missing pieces visible.*

Source: April 2026 operations dashboards (restricted; digitized aggregates in data/processed); docs/technical/scenario_methodology.md §3.

# How busy should a constructed evening be?

**We estimated a workload from April's completed rides, request-to-completion ratio and reported vehicle availability.**

**Completed rides, 6 pm hour**

≈ 5,100 rides ÷ 30 days

**≈ 170 rides / evening**

DERIVED

→

**Requests per completed ride**

50,297 requests ÷ 24,095 completed

**≈ 2.09 requests / ride**

DERIVED

→

**Requests in that hour**

170 rides × 2.09

**≈ 355 requests / hour**

DERIVED + ASSUMED

→

**Per vehicle on duty**

355 ÷ 27.6 vehicles

**≈ 12.85 requests / vehicle-hour**

DERIVED + ASSUMED

ASSUMED

This assumes the monthly request-to-completion ratio applies within the hour, and uses vehicle availability reported across services. It is an estimated workload, not observed arrivals.

**Five vans × hourly load → 64 / 50 / 54 requests in three hours = 168**

Five vans is our planned evening (a team design choice). Each hour uses its own rides and vehicles on duty. Illustrative range about 150–260 (unknown Homewood share of vehicles, repeat requests).

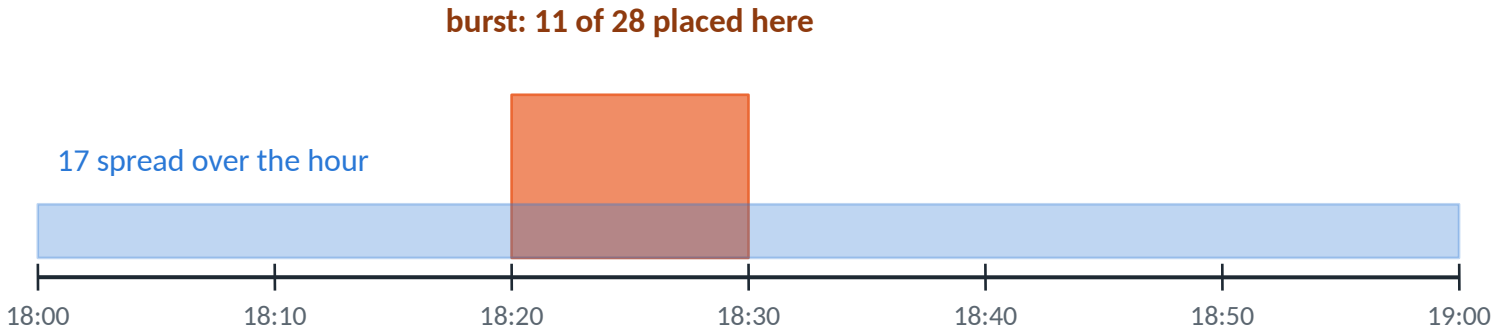
PLANNED — NOT YET RUN

The five-van, 168-request evening is designed but has not been run.

Source: April dashboards (rides by hour, status totals, vehicles on duty); docs/technical/decision_log.md D-35, D-36, D-49.

# We started with a smaller, deliberately demanding hour

## The current results come from two vans and 28 requests in one hour, repeated across 30 constructed nights.




Arrivals 18:00–19:00. Background requests can also fall in the burst window, so 11–17 requests (mean 13.7) arrive in 18:20–18:30 on a night; 34–51 passengers per night.

**This is a controlled pilot, not a reconstruction of a typical April night.**
Its load is 14 requests per van-hour, close to the 12.85 estimated for 6 pm (per vehicle reported across all services), but the burst deliberately concentrates demand, and two vans are a small fleet. It is designed to stress the transfer decision.

**2 vans**
14 seats each

**28 requests**
in one hour

**30 nights**
distinct constructed nights

**× 6 rules**
transfer rules, defined on slide 17

TESTED IN SIMULATION

*"Tested in simulation" means run in our model, not validated against Hopkins operations.*

Source: data/scenarios/v2/L5 (frozen inputs); docs/technical/scenario_spec.md §8; decision_log.md D-56, D-57.

# Knowing the endpoints does not identify the journeys

## Two different sets of trips can produce exactly the same pickup and drop-off totals.

### Trips, version 1

| | to X | to Y | total |
|---|---|---|---|
| **from A** | 2 | 0 | **2** |
| **from B** | 0 | 2 | **2** |
| **total** | **2** | **2** | **4** |

### Trips, version 2

| | to X | to Y | total |
|---|---|---|---|
| **from A** | 1 | 1 | **2** |
| **from B** | 1 | 1 | **2** |
| **total** | **2** | **2** | **4** |

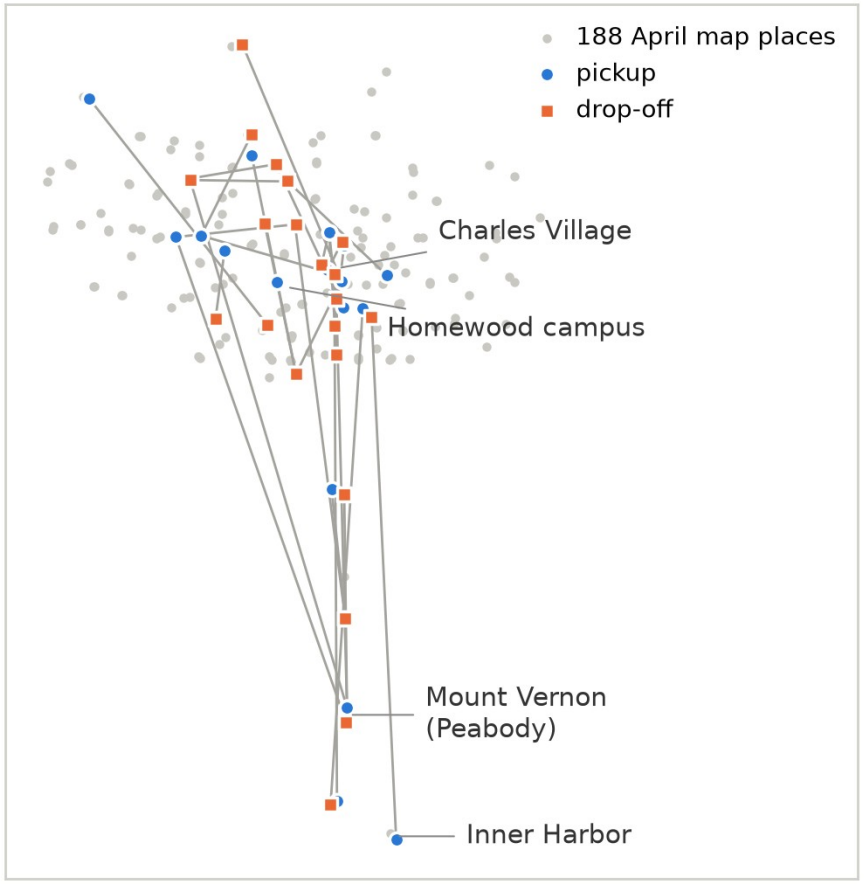
Green cells are all the maps show. Both versions match them exactly.

ASSUMED

**We assumed nearby journeys were more likely, then balanced the synthetic trips to preserve the observed location totals.**

- Distance preference: each extra minute of driving multiplies a pair's weight by about 0.9 (assumed rate 0.1 per minute)
- Balancing: rows and columns rescaled in turn until both April location totals match

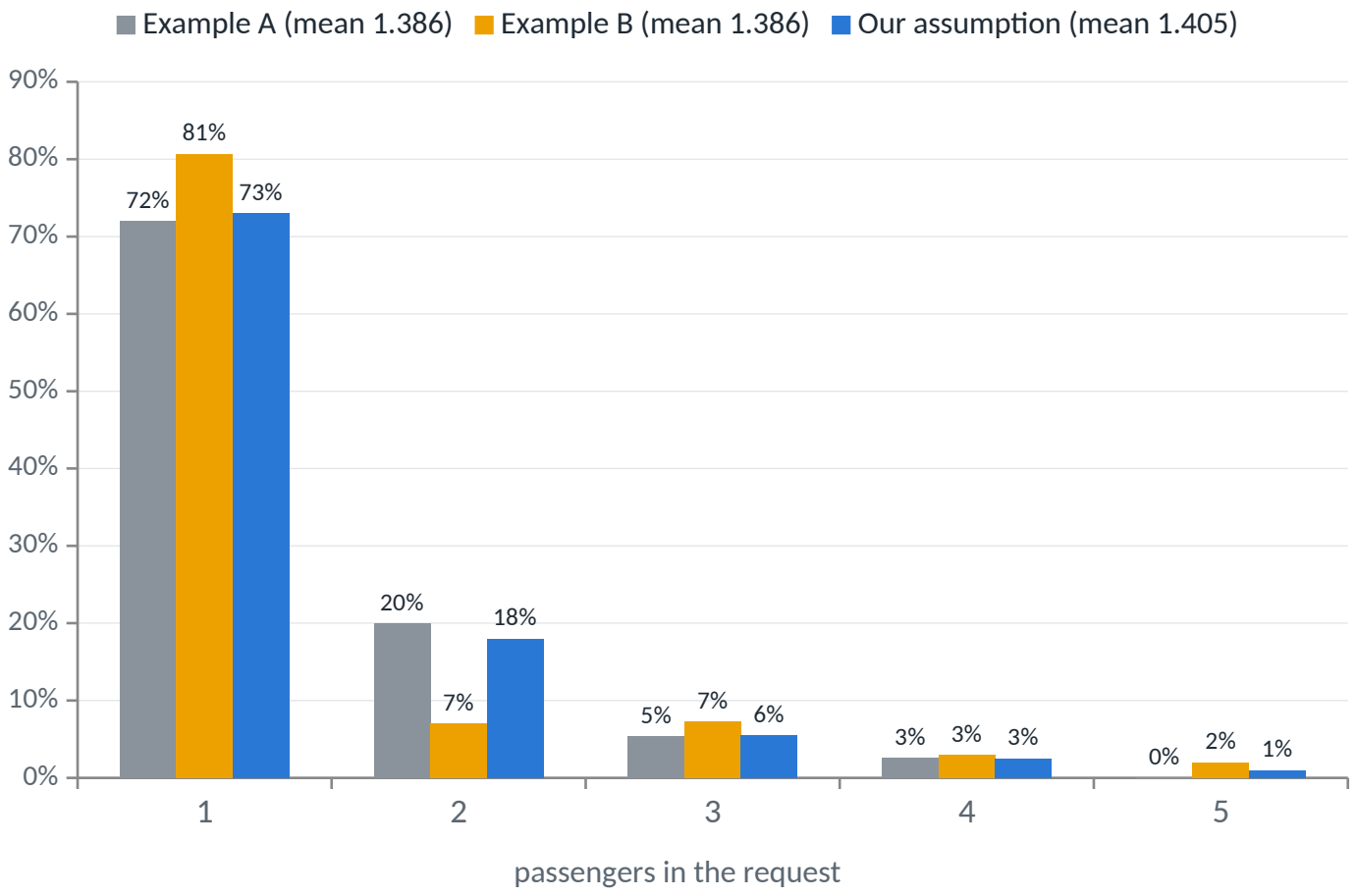
One constructed hour: 28 requests




Source: April pickup and drop-off maps (place totals); docs/technical/decision_log.md D-18; map: the first constructed pilot night.

# One average can hide very different groups

## April helps identify an average group size. It does not tell us the full distribution of groups.




DERIVED

**1.386**

van passengers per completed van request (ratio of two observed totals)

Requests with 3 or more passengers: Example A 8%, Example B 12.3%, ours 9%. Examples A and B are illustrations, not data.

ASSUMED

- Our chosen group-size mix is an assumption informed by the average—not a distribution recovered from April. It covers all requests, so its mean (1.405) need not equal 1.386
- Groups matter: seats per van, and waiting counted per passenger (slide 16)
- A party of five transferred to Lyft rides together in one XL (team decision)

Source: April dashboards (passengers, completed van requests); docs/technical/decision_log.md D-38, D-33. Bar labels rounded.

# One status label hides different events

**‘Cancelled’ does not tell us whether a rider gave up or a dispatcher moved the request to Lyft.**

## Observation

OBSERVED

23,843 of 50,297 requests (47%) ended “Canceled”. The label alone does not say why.

## Assumed status mapping

ASSUMED

We assume requests moved to Lyft also close as “Canceled” in the van system. The share that is riders giving up is kept as a range (50–75%) to test, not a fixed number.

## Hidden rider patience

ASSUMED

Each simulated rider gets a private deadline the dispatcher cannot see. Placeholder: median 18 minutes, waiting-dependent (Weibull).

**The patience distribution is provisional. It lets the simulator represent abandonment; it is not a behavioural distribution measured from April.** Many different patience distributions could produce the same cancellation share, so a chosen share does not pin one down.

Source: April dashboard status totals (digitized sums also show 99 “Denied”); docs/technical/decision_log.md D-39, D-49; methodology Appendix C.

# The simulator needs a probability at vehicle arrival

**The share of all requests marked ‘No Show’ is different from the chance of a no-show when a vehicle actually arrives.**

### Divide by all requests

$$\frac{2{,}359}{50{,}297} = 4.7\%$$

includes requests no vehicle ever reached

DERIVED

### Divide by van arrivals (completed + no-show)

$$\frac{2{,}359}{26{,}454} = 8.9\%$$

a no-show can only happen when a vehicle arrives

DERIVED + ASSUMED

The simulator gives each rider one presence draw, used whichever vehicle arrives (van or Lyft).

**8.9% holds only if:**
“No Show” was recorded after a real van arrival; “Completed” counts van trips; and Lyft hand-offs sit in “Canceled”.
**Our arrival-based probability is provisional because it depends on how the dashboard statuses were recorded.**

Source: April dashboard status totals; docs/technical/decision_log.md D-40, D-48; scenario_methodology.md Appendix C.

# The evidence becomes a scenario blueprint

**A scenario combines evidence-based inputs, explicit assumptions and hidden rider behaviour.**

## Demand

Arrival times

DERIVED + ASSUMED

Locations

DERIVED + ASSUMED

Trip pairing

ASSUMED

Party sizes

DERIVED + ASSUMED

## Resources

Vans and duty times

DESIGNED

Capacity (14 seats)

DESIGNED

Travel times × 1.2

DERIVED + ASSUMED

## Behaviour (hidden)

Patience

ASSUMED

Presence at the curb

DERIVED + ASSUMED

Lyft delay

ASSUMED

## Experiment settings

Van dispatcher (fixed)

DESIGNED

Transfer rule (varied)

DESIGNED

Ordering lag 2 min

ASSUMED

**Inputs are fixed before the run; outcomes are not.** Cancellations, no-shows and transfers happen during the run, from the hidden draws meeting the rule and the vans. They are results, not counts imposed in advance.

Source: docs/technical/scenario_spec.md v0.8 §3–§5; decision_log.md D-18–D-56.

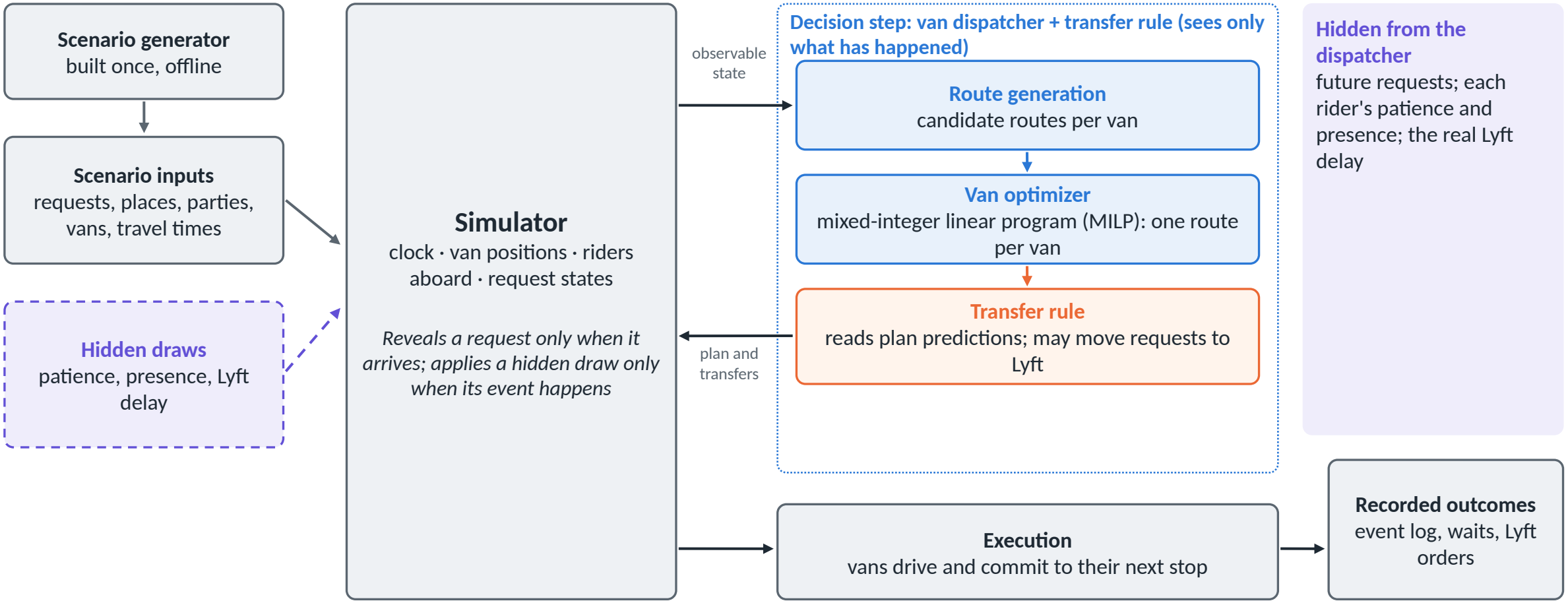
Where each input comes from: B4 →

# Three components have different jobs

**The simulator advances time and reveals events.**

**The van optimizer chooses among feasible candidate plans.**

**The transfer rule decides whether an eligible request moves to Lyft.**

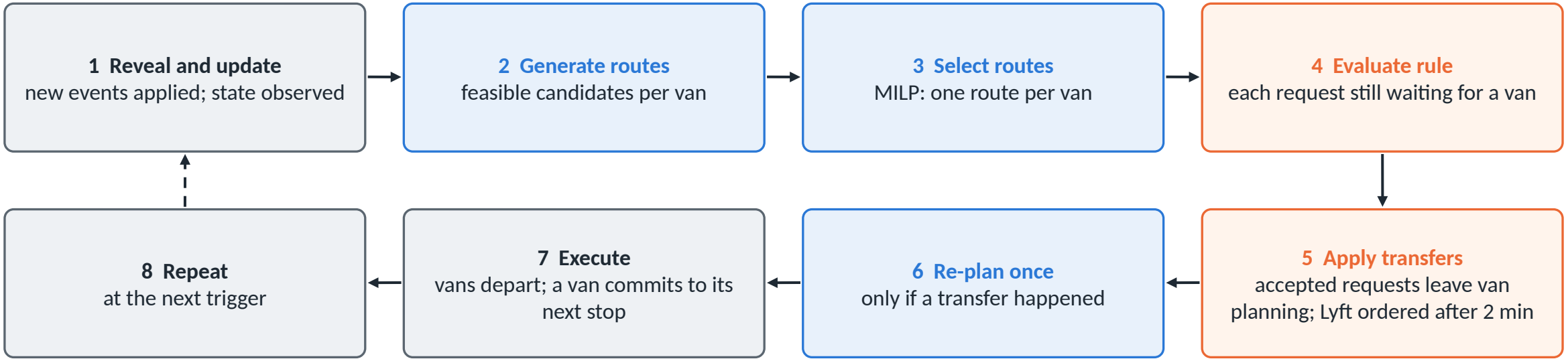



Source: src/sim/engine.py, dispatch.py, lyft.py; docs/technical/scenario_spec.md v0.8 §17.

Event order: B3 →

# The same decision cycle repeats as the evening unfolds

## The dispatcher sees requests that have arrived. It does not see future requests.




**When it runs:** every 2 minutes, and at a new request, a cancellation or no-show while waiting for a van, or a van coming on duty

**Transfers:** a transfer offer is accepted with a set probability (1.0 in the pilot); a declined request is not offered again

**No cascades:** the rule runs once per decision moment; after transfers the optimizer re-plans once, and the rule is not re-run in that second

**Commitments:** a stop a van is already driving to is fixed; requests committed to a van are not eligible for transfer

Source: src/sim/engine.py (epoch and triggers); docs/technical/scenario_spec.md v0.8 §17; decision_log.md D-46, D-52, D-55.

# The optimizer compares the consequences of candidate plans

## Waiting, riding, driving and unresolved requests enter the model in different ways.

### Waiting

**party size × φ(wait)**

φ is convex: 1 point per minute for the first 5 min, then 2.5, 5, 10, 20 after 5, 10, 15, 20 min

+

### Riding

**0.5 × party size × ride minutes**

a riding minute always counts less than a waiting minute

+

### Driving

**0.1 × van kilometres**

a tie-breaker

+

### Unresolved (postponed)

**postponement price + large constant**

the plan first serves as many requests as its candidates allow

Hard constraints: seats; pickup before drop-off; one route per van; each request served once or postponed. Ride-time guardrail max(25 min, 2 × direct + 5): no plan may create or worsen an excess (an unexpected no-show wait can still push a ride over).

ILLUSTRATIVE One van; rider 1 has waited 8 min; a party of two has waited 1 min

| Candidate | Waiting | Riding | Driving | Points |
|---|---|---|---|---|
| Route A: rider 1 first | 52.5 | 13.0 | 1.0 | **66.5 ✓ chosen** |
| Route B: party of two first | 67.5 | 12.5 | 1.0 | **81.0** |

In Route B rider 1 would wait 16 min; past 15 min each minute costs 10 points per passenger.

Formulation: B2 →

**'Optimal' means best among the candidate plans generated at that decision moment—not the best possible whole evening.**
Candidates: each van takes at most 3 new pickups, each set in its best stop order. Points are priorities, not money.

Source: docs/technical/model_design.md v0.5 §3–§4; scenario_methodology.md §6 (worked example; boarding time omitted).

# We changed the transfer decision, not the van dispatcher

| Rule | What it reads | Moves the request to Lyft when |
|---|---|---|
| **Vans only** | — | never |
| **Wait ten minutes** | time already waited | now − request time ≥ 10 min |
| **Wait fifteen minutes** | time already waited | now − request time ≥ 15 min |
| **Unassigned for ten minutes** | time continuously outside every van plan | ≥ 10 min (timer resets when the request enters a plan) |
| **Predicted total wait exceeds fifteen minutes** | predicted pickup clock time − request clock time | > 15 min |
| **Lyft arrives sooner** | predicted van pickup vs expected Lyft pickup | van predicted later than now + 2 lag + 8 typical + 2 margin |

grey: waiting-time rules · green: continuously unassigned time · blue: the predictive rules (they read the van plan's prediction)

**The six thresholds were specified before any run (spec §6) and not changed after seeing results.**

**Predicted total wait**
= predicted pickup clock time − request clock time

Predicted pickup: the van plan's time, or a tentative estimate if the request is postponed; no feasible van counts as an infinitely late pickup, so the rule fires.

**Eligible requests**
still waiting for a van, not committed to one, and never having declined a Lyft offer. Evaluated once per decision moment.

Code key: B1 →

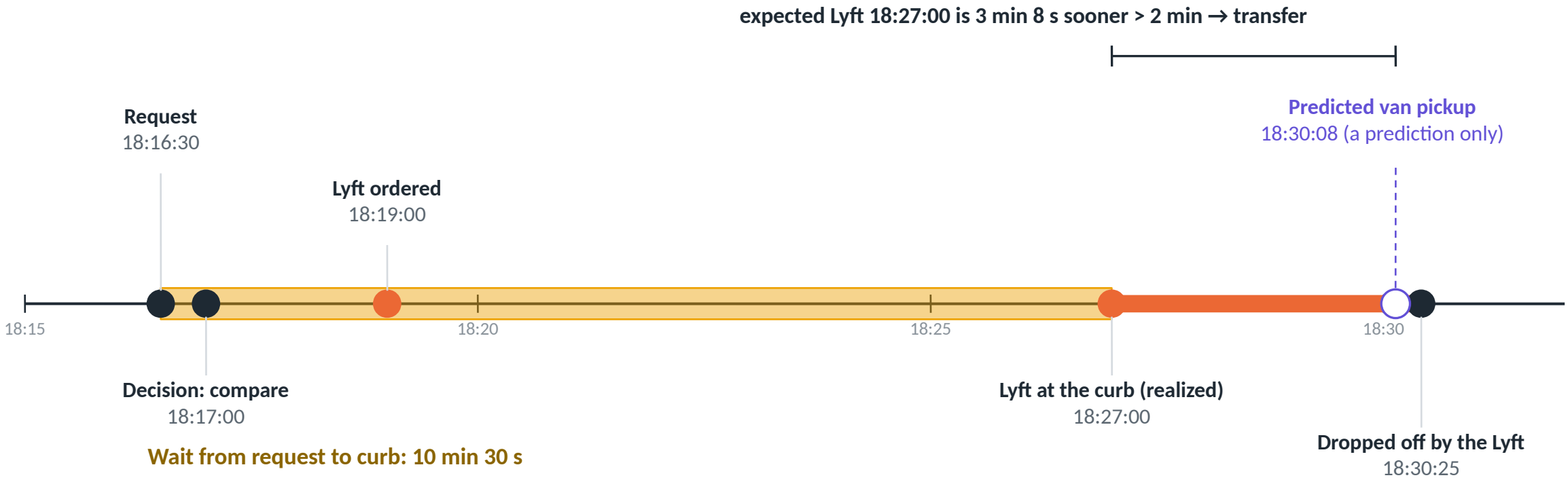
Source: src/sim/lyft.py; engine.py:305-330; docs/technical/scenario_spec.md v0.8 §6–§7; preregistration_v2.md (draft).

# What does ‘Lyft arrives sooner’ actually compare?

## The rule compares predicted van pickup with expected Lyft pickup, including the ordering delay.

DESIGNED

A small hand-built evening, separate from the randomized pilot. Here the Lyft delay is fixed, so expected and realized Lyft times coincide by design.



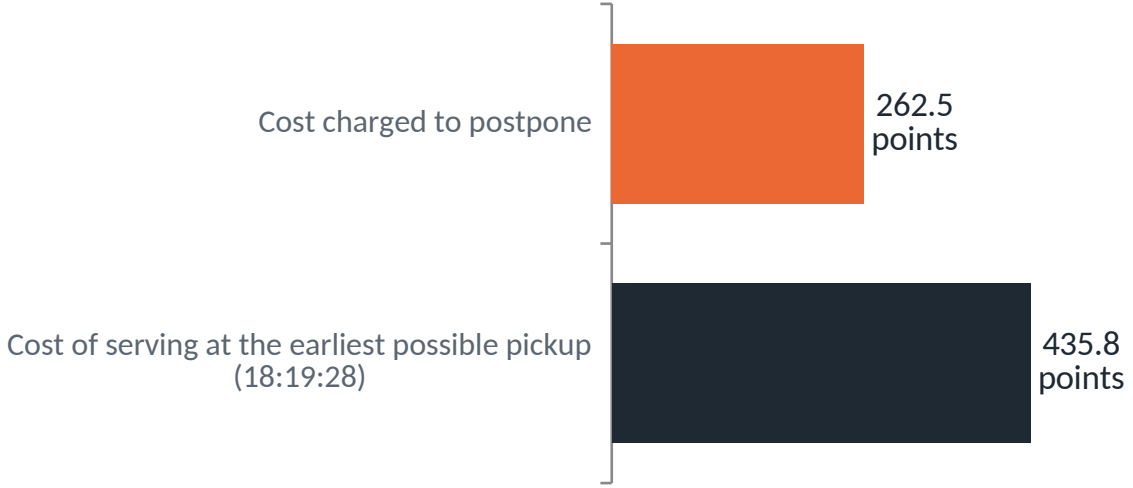

Expected Lyft pickup = 18:17:00 + 2-min ordering lag + 8-min typical delay = 18:27:00. The van pickup time is a prediction. We do not observe what that van would have done after choosing a different action.

Source: outputs/sim/L4_R3-2 (epochs.jsonl, outcomes.csv: request R011); docs/technical/scenario_spec.md §6.

# A cheap postponement price could keep a rider waiting

## The first formulation could price postponement more cheaply than a physically possible pickup.

**1 The flaw: at 18:15 on the designed evening, a party of five had waited 15 min**




Postponing looked cheaper than any service a van could really give, so a long-waiting party could be left waiting.

**2 Controlled comparison: the whole designed evening, vans only, three formulations (all three in B5)**

| | First version | Adopted |
|---|---|---|
| Worst wait | 44.4 min | **35.6 min** |
| Mean wait | 18.8 min | 18.8 min |
| Median wait | 17.1 min | 18.6 min |

**The adopted approximation reduced the worst wait in this designed case. It did not reduce average waiting.**

**3 Adopted (provisionally):** price postponement at the earliest time any on-duty van could physically reach the request. It changes cost coefficients only, adding no variables.

**4 Still optimistic:** it ignores the work the nearest van is already doing. One designed evening does not show a general improvement.

Three formulations: B5 →

Source: outputs/sim/d53_experiment; docs/technical/decision_log.md D-53; model_design.md v0.5 §3.5. Provisional choice.

# Replay the same night; change one rule

**Each rule faces the same requests and hidden rider inputs, so differences reflect the rule within this specified model.**

## One constructed night

same requests, places and parties
same hidden patience and presence
same Lyft delay per rider
same vans

- Vans only
- Wait ten minutes
- Wait fifteen minutes
- Unassigned for ten minutes
- Predicted total wait exceeds fifteen minutes
- Lyft arrives sooner

**Thirty nights replayed six ways are thirty paired scenarios—not 180 independent nights.**

Pairing removes night-to-night differences from each comparison. It does not make the nights representative of April.

TESTED IN SIMULATION

Provisional settings: 2-min ordering lag; every Lyft offer accepted; 8.9% no-show per arrival; placeholder patience; van travel × 1.2.

Source: docs/technical/decision_log.md D-56, D-57; outputs/sim/batch_L5_v1/summary.md.

# We checked implementation before interpreting outcomes

**Verification asks whether the code implements the specified model. Validation asks whether the model represents the real service.**

## Verification evidence

| | |
|---|---|
| **to the second** | 3 small designed evenings reproduce hand-computed event times |
| **292,181** | plan combinations on a designed evening, enumerated by a separate brute-force program over the same candidates; always the same choice |
| **253 tests** | pass; 168 written by separate AI review agents that did not reuse the simulator's code |
| **6 of 6** | sampled reruns byte-identical |

## Not validation

- Do riders behave like our hidden draws?
- Do our evenings resemble real Homewood nights?
- Would the rules behave the same in real operations?

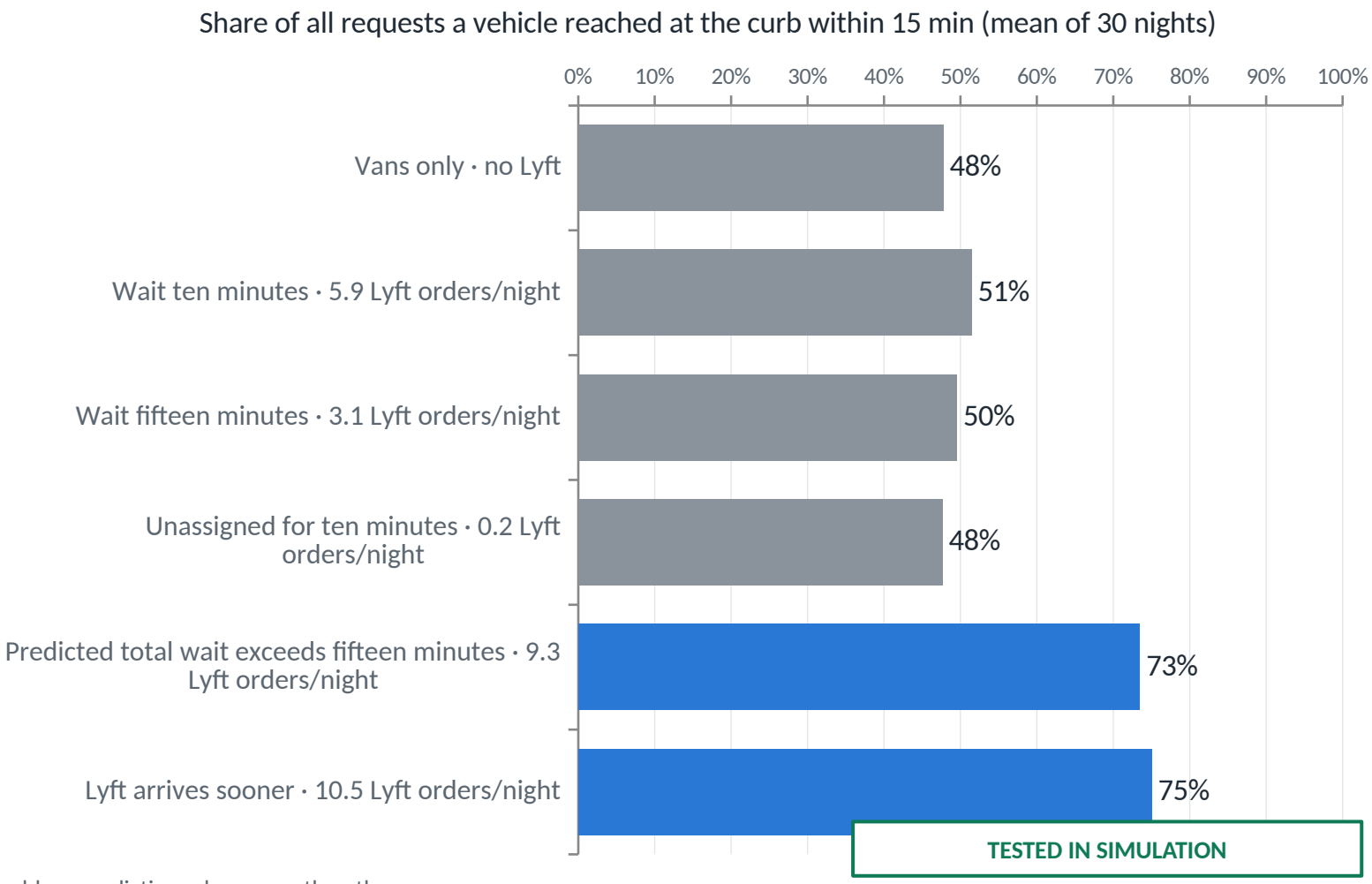
**AI wrote the simulator code and the checks, under our direction. Separate implementations are useful checks, but they share our assumptions and do not replace our own understanding of the model.**

Diagnostics: B8 →

Source: Phase 4 log; tests/; outputs/sim/batch_L5_v1/reproducibility.json; docs/ai_use_log.md.

# The predictive rules reached more requests promptly—and used more Lyft

**30 constructed burst-demand nights; provisional rider behaviour.**




blue: predictive rules · grey: the others

In this two-van pilot, the predictive rules reached about three quarters of requests within fifteen minutes, compared with about half under vans only. They also used roughly nine to ten and a half Lyft orders per night.

**Metric: a vehicle reaching the curb—not boarding or a completed trip.** A rider moved after 10–15 min of waiting almost never reaches the curb within 15 min, so this measure favours early transfer.

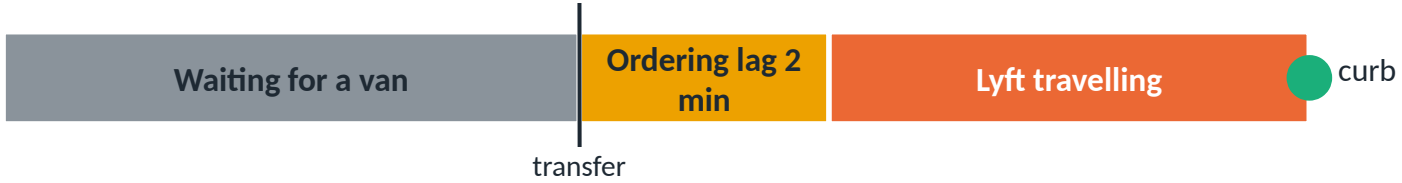
**This comparison does not isolate the benefit of prediction at an equal Lyft budget.**

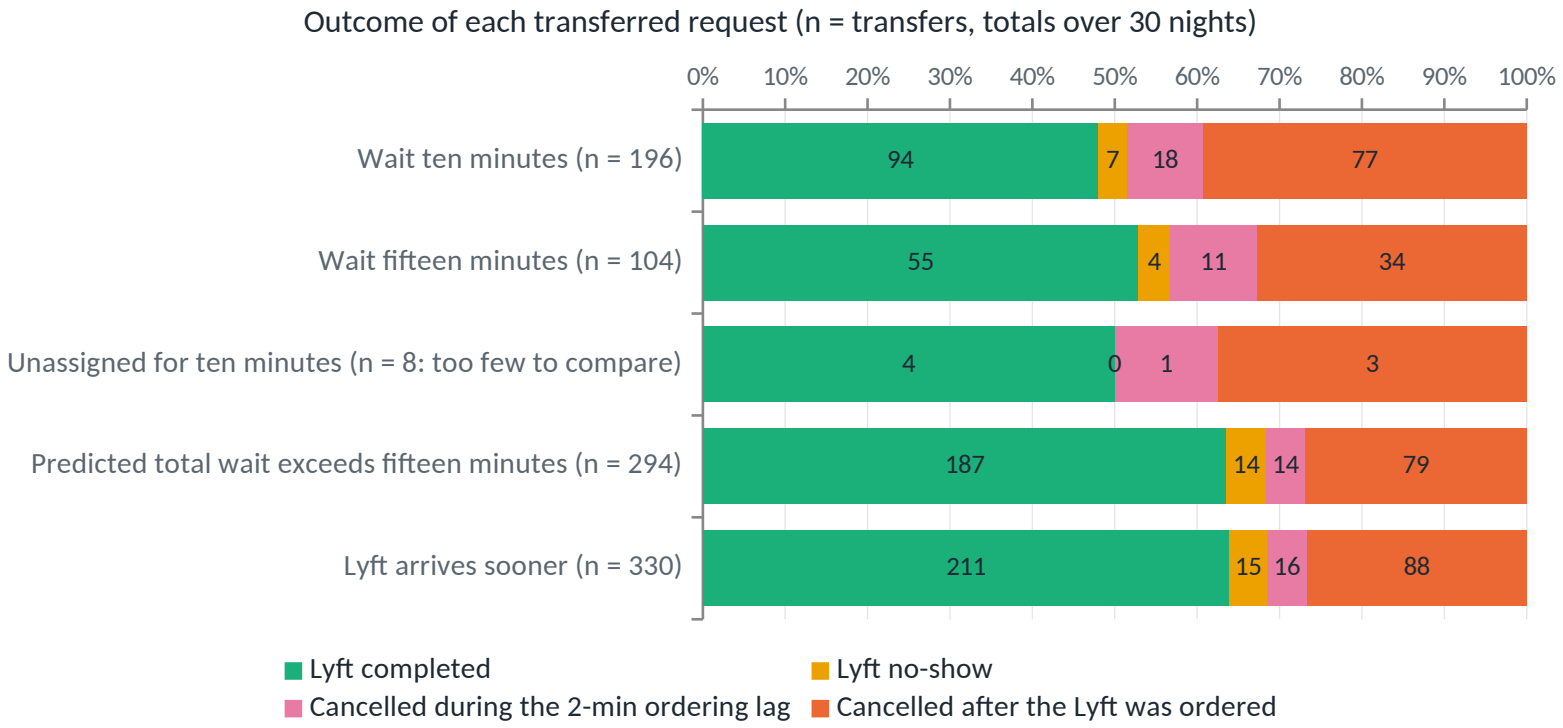
All metrics: B6 →

Source: outputs/sim/batch_L5_v1 (tradeoff.md, summary.md). Descriptive means; between-night SD 0.07–0.10.

# Moving a request to Lyft does not end its wait

## The rider still waits through ordering and Lyft arrival. Their original patience clock continues running.




**95 of 196**

transferred under "Wait ten minutes" cancelled before the Lyft arrived; about a third under the prediction rules

Riders moved by the prediction rules had usually waited less when transferred. Under our placeholder patience, that is consistent with their lower share lost after transfer: a possible contributing mechanism, not an isolated cause.

ASSUMED

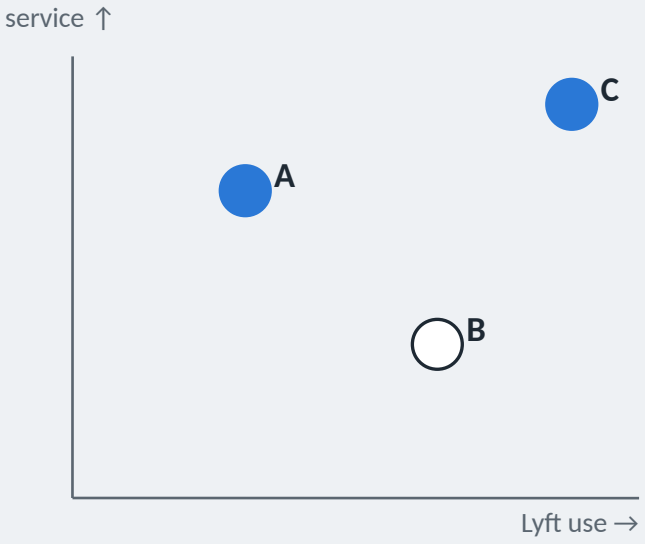
Whether the gap between policies survives other patience shapes is untested.

Source: outputs/sim/batch_L5_v1 per-run outcomes; transfers = Lyft orders + cancellations during the ordering lag.

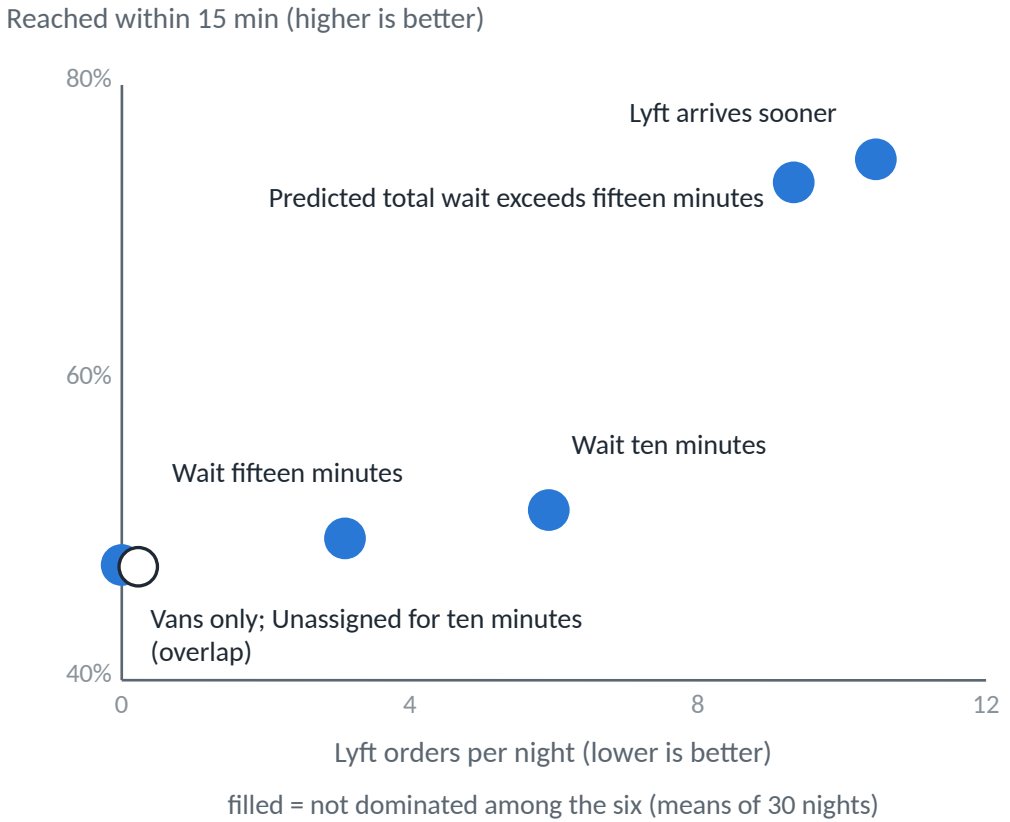
# Better service and lower Lyft use can point to different choices

## A tested rule is dominated if another tested rule is at least as good on both selected objectives and strictly better on one.

### Dominance, in one picture




A dominates B. A and C: neither dominates the other.




filled = not dominated among the six (means of 30 nights)

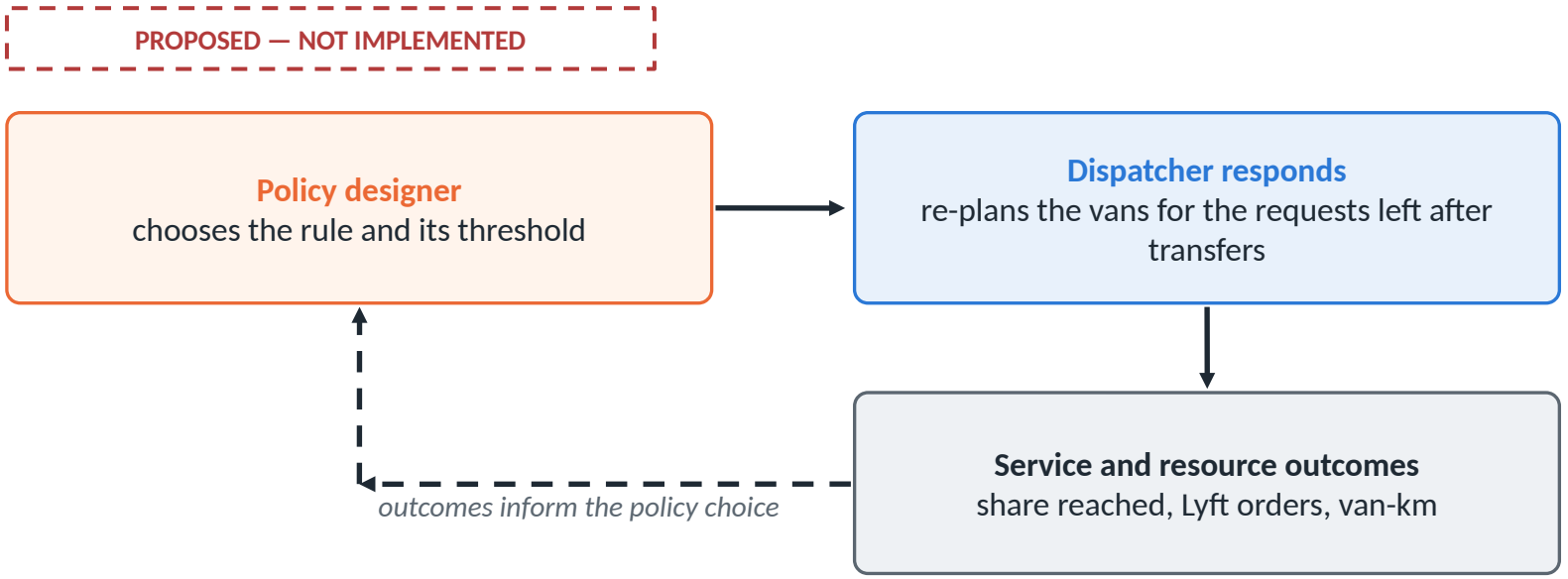
- Nondominated does not mean recommended.
- Vans only stays nondominated through zero Lyft use.
- “Unassigned for ten minutes” is dominated on the means: service within about one request in 840 of vans only, slightly more Lyft (0.23 vs 0 a night). No noise margin applied.
- This is the nondominated set among six tested rules under these two objectives—not the complete frontier of possible policies.
- Adding objectives or decision criteria can change the assessment.

Night by night: B7 →

Source: src/sim/tradeoff.py; outputs/sim/batch_L5_v1/tradeoff.md. Descriptive, not a ranking; decision criteria (D-50) not agreed.

# Could we choose the transfer policy systematically?

## One possible extension is to optimize transfer policy while anticipating how the routing dispatcher responds.




**Today:** six fixed rules compared on the same nights. A finite search finds the best option in a finite set for the average over our sampled nights —not the true expectation, not continuous thresholds, not all policies.

**To state this precisely as a leader–follower (bilevel) model, we would still need to define:**

1. the leader's choices (rule family, threshold range)
2. the follower: one decision moment at a time, or the whole evening
3. how ties in the follower's best response are treated
4. the expectation over constructed nights
5. the loss and the weight on Lyft use
6. any budget or service-floor constraints

Source: docs/technical/scenario_methodology.md §7b; Colson, Marcotte & Savard (2007), Ann. Oper. Res. 153:235–256; Dempe (2002).

# Which assumptions could change the answer?

**The findings are conditional on the evenings and behaviours we constructed.**

| Assumption | Status | How it could change the interpretation |
|---|---|---|
| Rider patience | ASSUMED placeholder | The assumption we most expect to matter (untested): it sets how many riders give up and the losses after transfer |
| Lyft delay | ASSUMED | A slower or more variable Lyft could weaken transfers; under our placeholder patience, late transfers most |
| Workload and burst | DERIVED + ASSUMED | The pilot is deliberately heavy; at lighter load the rules may barely differ |
| Geography and pairing | ASSUMED | Longer or more scattered trips change van reach and predicted pickups |
| Candidate restriction | DESIGNED | At most 3 new pickups per van per decision; a richer search could change van performance and the rules' predictions |

Also open: the 15-minute measure favours early transfer; 8.9% no-show depends on the status mapping; two vans, not five; verification, not validation.

Source: docs/technical/scenario_methodology.md §7–§9; decision_log.md D-39, D-43, D-49, D-52.

# Test robustness before fine-tuning a winner

**Next, we should test whether the findings survive different rider-patience and Lyft-delay assumptions, while checking feasibility at the larger fleet size.**

### 1 Behavioural sensitivity

Rerun the saved nights under other patience shapes and Lyft delays

PLANNED — NOT YET RUN

### 2 Five-van feasibility

Run the 168-request evening; check solve times and candidate limits

PLANNED — NOT YET RUN

### 3 Agreed evaluation criteria

Decide how much extra Lyft use one more rider reached is worth, then compute confidence intervals across the paired nights

PLANNED — NOT YET RUN

### 4 Threshold / policy search

Only then sweep thresholds, or pursue the leader–follower extension

PLANNED — NOT YET RUN

**Order matters: a threshold tuned to placeholder behaviour could be tuned to our assumptions rather than to the service.**

Source: docs/technical/scenario_methodology.md §8–§9; decision_log.md D-50, D-51.

# What we can defend today

**Aggregate data did not give us individual journeys. It gave us constraints for building plausible evenings.**

Within our specified pilot, predictive transfer rules improved timely curb arrival while using more Lyft. Choosing a policy requires testing the assumptions and deciding what trade-off is acceptable.

TESTED IN SIMULATION

in simulation, on 30 constructed two-van nights · not a policy recommendation

Source: this deck, slides 6–27.

Backups →

# Backup slides

B1 Code key · B2 Route-selection MILP · B3 Event order · B4 Where inputs come from · B5 Three postponement prices · B6 All metrics by rule · B7 Night by night · B8 Numerical diagnostics · B9 Corrected proposal claims

B1 B2 B3 B4 B5 B6 B7 B8 B9

# B1  Code key (files and questions only)

| Said in the talk | Code in files and logs |
|---|---|
| Vans only | R0 |
| Wait ten minutes | R1(10) |
| Wait fifteen minutes | R1(15) |
| Unassigned for ten minutes | R1u(10) |
| Predicted total wait exceeds fifteen minutes | R2(15) |
| Lyft arrives sooner | R3(2) |
| Small designed evenings with hand-computed answers | L1–L4 (L4: vans from 18:15, ordering lag 2 min, Lyft delay fixed 8 min, acceptance 0.9) |
| The two-van pilot | L5: 30 constructed one-hour nights, 2 vans, 28 requests |
| The van optimizer | Dispatcher C: restricted route-selection MILP (HiGHS) |
| The adopted postponement price | Pending price A (earliest physical pickup) |
| Lyft order; ordering lag | “booking” and “booking lag” in files (D-43) |
| Numbered decisions | D-xx entries in docs/technical/decision_log.md |

Source: src/sim/lyft.py; scenario_spec.md §6, §8; decision log.

← back to slide 17

# B2 The route-selection MILP at one decision moment

$$\min \ \Sigma_v \Sigma_k c_{vk} y_{vk} + \Sigma_r \pi_r u_r$$

$$\text{s.t.} \ \Sigma_k y_{vk} = 1 \quad \text{for every van } v \qquad \text{(one route per van)}$$

$$\Sigma_{(v,k):\, r \in S_{vk}} y_{vk} + u_r = 1 \quad \text{for every waiting request } r \quad \text{(served once, or postponed)}$$

$$y, u \in \{0, 1\}; \quad S_{vk} = \text{requests picked up by candidate route } k \text{ of van } v$$

- $c_{vk}$: route points (waiting $\varphi$ with slopes 1/2.5/5/10/20 per passenger-minute after 0/5/10/15/20 min; riding 0.5 per passenger-minute; 0.1 per km; $\Psi$ only for unavoidable existing guardrail excess); exact integers
- Candidates: every subset of at most 3 waiting requests per van, each in its best stop order (exact search); the van's committed work is always a candidate
- $\pi_r = b_r + M$; $b_r$ from the adopted postponement price; $M = 1 + \Sigma_v (\max_k c_{vk} - \min_k c_{vk}) + \Sigma_r b_r$: every proven-optimal solution postpones the fewest requests the candidates allow
- Ride-time guardrail max(25, 2 × direct + 5) min: no candidate may create or worsen an excess
- HiGHS: relative gap 0, absolute gap 0.5 units; ties broken by a fixed canonical order; a failed solve is logged, never replaced by a heuristic

Source: model_design.md v0.5 §3; decision log D-47, D-52 (amendments), D-53.

← back to slide 16

# B3 Event order and a request's life

- Same-second order: drop-offs → curb arrivals (stop the patience clock) → service ends → vans on/off duty → new requests → patience checks → Lyft orders → decision moment → departures
- Decision-moment triggers in the code: new request; cancellation or no-show while waiting for a van; van coming on duty; the 2-minute tick (van going off duty does not trigger one; Lyft events never do)
- A van commits to a stop when it departs toward it; a committed request is not eligible for transfer
- After a transfer: a 2-min ordering lag, then the rider's hidden Lyft delay; the rider may cancel at any point before the Lyft reaches the curb
- During an absent rider's 3-min curb wait, planning assumes possible boarding at expiry; the simulator never boards an absent rider
- Presence is one draw per rider, used whichever vehicle arrives; planning takes zero simulated time

Source: src/sim/engine.py; scenario_spec.md v0.8 §17; decision log D-46, D-48, D-54.

← back to slide 14

# B4 Where each input comes from

| Input | Status | Value | Basis |
|---|---|---|---|
| Requests per hour | DERIVED + ASSUMED | 64 / 50 / 54 for five vans (planned); pilot: 28 in one hour, two vans | D-35, D-36, D-56 |
| Trip pairing | ASSUMED + fitted | Distance prior, rate 0.1 per min, balanced to both map totals | D-18 |
| Party size | ASSUMED (mean derived) | Shares 1–5: .73/.18/.055/.025/.01; April 1.386 van passengers per completed van request | D-38 |
| Patience | ASSUMED placeholder | Weibull, shape 1.5, median 18 min (scale ≈ 23.0 min) | D-39 |
| No-show | DERIVED, conditional | 8.9% per vehicle arrival if the status mapping holds (4.7% of all requests) | D-40, D-48 |
| Lyft | ASSUMED | Ordering lag 2 min; delay lognormal, median 8 min, 90th percentile 15 min | D-43 |
| Travel times | DERIVED; × 1.2 ASSUMED | Local OSRM routing on OpenStreetMap; van times × 1.2 in the pilot (placeholder) | D-31, D-47 |

Source: decision log D-18–D-56; methodology v1.5 §4.

← back to slide 13

# B5 Three postponement prices compared

| | First version | Adopted (earliest physical pickup) | Route-based |
|---|---|---|---|
| Price of postponing a request | as if served at the next 2-min re-plan | as if served at the earliest time any on-duty van can reach it | as if served after a chosen van's route |
| Wait median / 90th pct / worst (min) | 17.1 / 31.0 / 44.4 | 18.6 / 31.2 / 35.6 | 21.1 / 34.3 / 38.6 |
| Mean wait (min) | 18.8 | 18.8 | 21.8 |
| Party of five from slide 19: wait | 22 min 18 s | 19 min 28 s | 20 min 39 s |
| Extra variables | 0 | 0 | up to 9,828 |
| Total MILP solve time on this evening (s) | 0.1 | 0.1 | 4.8 |
| Direction of error | optimistic | optimistic (ignores the van's other work) | either direction |

Designed evening, vans only, one run per formulation. Adopted for its meaning and no added variables, provisionally; computing the earliest pickups is extra work per decision, not separately timed. None guarantees a feasible schedule for all waiting riders.

Source: outputs/sim/d53_experiment; decision log D-53; review d53_pending_price_review (22 Oct).

← back to slide 19

# B6  All six rules over 30 constructed nights

| Per night | Vans only | Wait ten minutes | Wait fifteen minutes | Unassigned for ten minutes | Predicted total wait exceeds fifteen minutes | Lyft arrives sooner |
|---|---|---|---|---|---|---|
| Reached within 15 min (share of 28) | 0.48 (0.09) | 0.51 (0.08) | 0.50 (0.09) | 0.48 (0.10) | 0.73 (0.07) | 0.75 (0.08) |
| Requests picked up (of 28) | 16.1 (2.4) | 17.2 (2.6) | 16.5 (2.6) | 16.2 (2.5) | 20.2 (2.0) | 20.4 (2.3) |
| Wait median, min (per request picked up) | 8.6 (1.8) | 8.3 (1.6) | 8.4 (1.5) | 8.9 (2.1) | 7.5 (1.0) | 7.3 (1.0) |
| Wait 90th pct, min (per request picked up) | 20.2 (5.3) | 18.1 (2.4) | 19.6 (4.0) | 20.2 (4.6) | 13.3 (1.5) | 12.2 (1.7) |
| Wait 90th pct, min (per passenger picked up) | 20.1 (5.8) | 17.4 (2.7) | 19.2 (4.8) | 19.6 (4.7) | 13.2 (2.3) | 12.7 (2.7) |
| Van completed | 16.1 (2.4) | 14.1 (2.3) | 14.6 (2.4) | 16.1 (2.5) | 14.0 (2.7) | 13.3 (2.6) |
| Lyft completed | 0.0 (0.0) | 3.1 (1.4) | 1.8 (1.2) | 0.1 (0.3) | 6.2 (1.9) | 7.0 (1.6) |
| Cancelled waiting for van | 10.8 (2.3) | 6.5 (2.3) | 8.9 (2.5) | 10.5 (2.4) | 3.3 (1.7) | 2.6 (1.5) |
| Cancelled after transfer | 0.0 (0.0) | 3.2 (1.8) | 1.5 (1.0) | 0.1 (0.4) | 3.1 (1.5) | 3.5 (1.5) |
| Lyft orders | 0.0 (0.0) | 5.9 (1.6) | 3.1 (1.4) | 0.2 (0.5) | 9.3 (2.5) | 10.5 (1.9) |
| Van-km | 51.2 (4.4) | 47.6 (5.5) | 48.6 (5.0) | 51.9 (4.8) | 45.7 (5.7) | 43.0 (4.0) |
| Run time, s | 43 (46) | 40 (44) | 49 (57) | 50 (56) | 5 (7) | 4 (5) |

Cells: mean of nightly values (standard deviation across nights). Waits cover only requests picked up. 30 nights replayed six ways, not 180 independent nights.

Source: outputs/sim/batch_L5_v1 (per_night_metrics.csv), via make_numbers.py.

← back to slide 22

# B7  Trade-off night by night

| Rule | Reached (mean) | Lyft orders (mean) | Nights not dominated | Nights identical to vans only | Nights reach above / below vans only |
|---|---|---|---|---|---|
| Vans only | 0.477 | 0.00 | 30 of 30 | 30 | 0 / 0 |
| Wait ten minutes | 0.514 | 5.93 | 11 of 30 | 0 | 19 / 6 |
| Wait fifteen minutes | 0.495 | 3.10 | 14 of 30 | 1 | 15 / 5 |
| Unassigned for ten minutes | 0.476 | 0.23 | 26 of 30 | 24 | 2 / 2 |
| Predicted total wait exceeds fifteen minutes | 0.735 | 9.33 | 24 of 30 | 0 | 30 / 0 |
| Lyft arrives sooner | 0.750 | 10.47 | 12 of 30 | 0 | 30 / 0 |

Ties: a rule that makes no transfer on a night replays vans only exactly; identical results do not dominate each other, so both count as not dominated. Above / below counts are paired comparisons on the same constructed night, with no intervals yet.

Source: outputs/sim/batch_L5_v1/tradeoff.md.

← back to slide 24

# B8 Numerical diagnostics (two-van pilot)

- 180 runs = 30 constructed nights × 6 rules; every run passed every per-run check; one code version for all runs
- 13,663 decision moments (epochs); the main solve was proven optimal over its candidates in all of them
- 41 decision moments had an incomplete search for equally good plans (a solver error); in all 41, full enumeration shows a unique optimum, which is the plan chosen
- 36 ride-time excess events across the 180 runs (16 distinct requests on 10 nights, repeated across rules); each coincided with a no-show wait on that van while the rider was aboard (attribution check, not proof); 0 unexplained
- Run time per run: mean 32 s, longest 253 s (Python, one core per run)
- Reproducibility: 6 of 6 sampled reruns byte-identical; the 2 first-night batch runs identical to the earlier pilot outputs

Source: outputs/sim/batch_L5_v1 (summary.md, tie_incomplete_check.csv, reproducibility.json); decision log D-52 am. 5, D-57.

← back to slide 21

# B9 Corrected proposal claims

| Proposal said | Corrected | Why |
|---|---|---|
| About 5,100 rides per hour at 6 pm | ≈ 170 completed rides in one evening's 6 pm hour | The bar is a 30-day total (tooltip 5,098; digitized 5,089) |
| About 28 vans in service | 27.6 vehicles on duty at 6 pm, “All services” average | Not a Homewood-only count |
| 7 pm bar: 4,900 | 3,957 (digitized 30-day total) | Proposal chart value differs from our digitized reading |
| ~1 in 4 trips overflow to Lyft | 24.5% of 44,374 riders rode Lyft | A share of riders, not of trips or requests |
| Trip-level extracts coming | Constructed evenings from aggregate dashboards | The extracts did not arrive |
| Compare dispatch methods | Hold one specified dispatcher fixed; vary the transfer rule | Scope choice: isolate one controllable decision |
| Lyft as paid overflow inside the optimizer | A separate, explicit transfer rule; no money modelled | A dispatcher sends the Lyft; cost per Lyft trip unknown (D-45) |
| Targets as hard constraints (20/25 min) | Reporting thresholds plus a ride-time guardrail | Target wording changed over time (15/20/20 around April; 20/25/25 on the 21 Oct page) (D-49) |

The September proposal deck is unchanged; these corrections are recorded in methodology v1.5 §1a.

Source: methodology v1.5 §1a; decision log D-25, D-35, D-36, D-45, D-47, D-49, D-51.

← back to slide 4